Blip is a lightweight JavaScript library that wraps the Web Audio API, and also provides some extremely powerful and flexible methods for looping and manipulating samples that allow for both temporal precision and musical expressiveness

like the ones for incoming chat on Facebook or an incoming call on Google Hangouts create an instant recognition in the user's mind of what is going on at the screen. But then why do we see such limited use of sound in the Internet? The reason is fairly simple, there has been a legacy of mostly miserable and unaesthetic use of sound like MIDIs and Flash Audios in the past that mostly discourage current UI developers from considering sound as a tool in their designs.

Getting started

Choose your areas and types of sounds you want to use carefully, and take precautions to not overuse it. Like any good design, it should enhance the user experience and not be annoying instead. Choose whether you want to use any pre-recorded audio or whether you want to generate audio at runtime. The Web Audio Interface is part of HTML5 and thus no new libraries are required to be included. Just a plain declaration of <!DOCTYPE html> is enough to get you started. The Web Audio API gives a very high level of control over sounds and has a total of 28 interfaces and associated events. However, if you are interested in JS based libraries to manipulate Web Audio then Blip and HowlerJS are great options to get started with.

In Web Audio the timing control has high precision and low latency, allowing us to write code that responds accurately to events and is able to target specific samples, even at a high sample rate. The Web Audio API also allows us to control how audio is spatialized. Using a system based on a source-listener model, it allows controlling the panning model and deals with distance-induced attenuation or doppler shift induced by a moving source (or moving listener) thereby providing for creation of effects such as "Surround Sound" or "Distance Specific Sounds".

The steps involved in creating the Audio context

- Create sources like <audio>, oscillator, stream etc.
- Create and apply effects nodes such as biquad filter, panner, reverb
- Choose a final destination for the audio
- Connect the sources up to the effects, and the effects to the destination.

Quick introduction to a few audio elements

The Web Audio APIs are vast and are made up of as many as 28 interfaces, but we shall highlight and use a few of them in our examples; here are some:

- **AudioNode:** Its a generic interface of audio processing modules like an audio source (e.g. an HTML <audio> or <video> element, an OscillatorNode, etc.), the audio destination, intermediate processing module (e.g. a filter like BiquadFilterNode or ConvolverNode), or volume control (like GainNode).
- **AudioContext:** The AudioContext interface can be visualized as the entire container for one complete sound processing unit built from linked audio modules, each represented by an AudioNode. There can be only one AudioContext per page.
- **AudioBuffer:** This interface holds any short audio asset residing in memory, created from an audio file or from raw data .
- **GainNode:** Used for Volume modulations, it is an AudioNode module that causes a given gain to be applied onto the input before it is sent to the output.

Let's try them out

We shall make a web application where we shall have two components.

First we shall play the Major notes of "Do Re Mi Fa So La Ti Do" or "Sa Re Ga Ma Pa Dha Ni Sa" with basic volume control.

Second, we are going to modify a javascript game modelled on the popular breakout/bricks game originally scripted by Andrzej Mazur. We shall play a single sound whenever the ball collides with a brick.

Step by step:

Let's create the AudioContext for the same which will act as the container for generating sounds:

```
var context = new (window.AudioContext || window.webkitAudioContext)();
// Webkit/blink browsers need prefix; Safari won't work without window.
// Note : The general format for WebContext is: var context = new AudioContext();
```

Now for the volume control let us create a gain node and connect it to the destination :

```
var gainNode = context.createGain();
gainNode.connect(context.destination);
```

Even though you may not be able to do this from the get go, if you play around with WebAudio long enough, eventually you'll get there